

Development Platforms

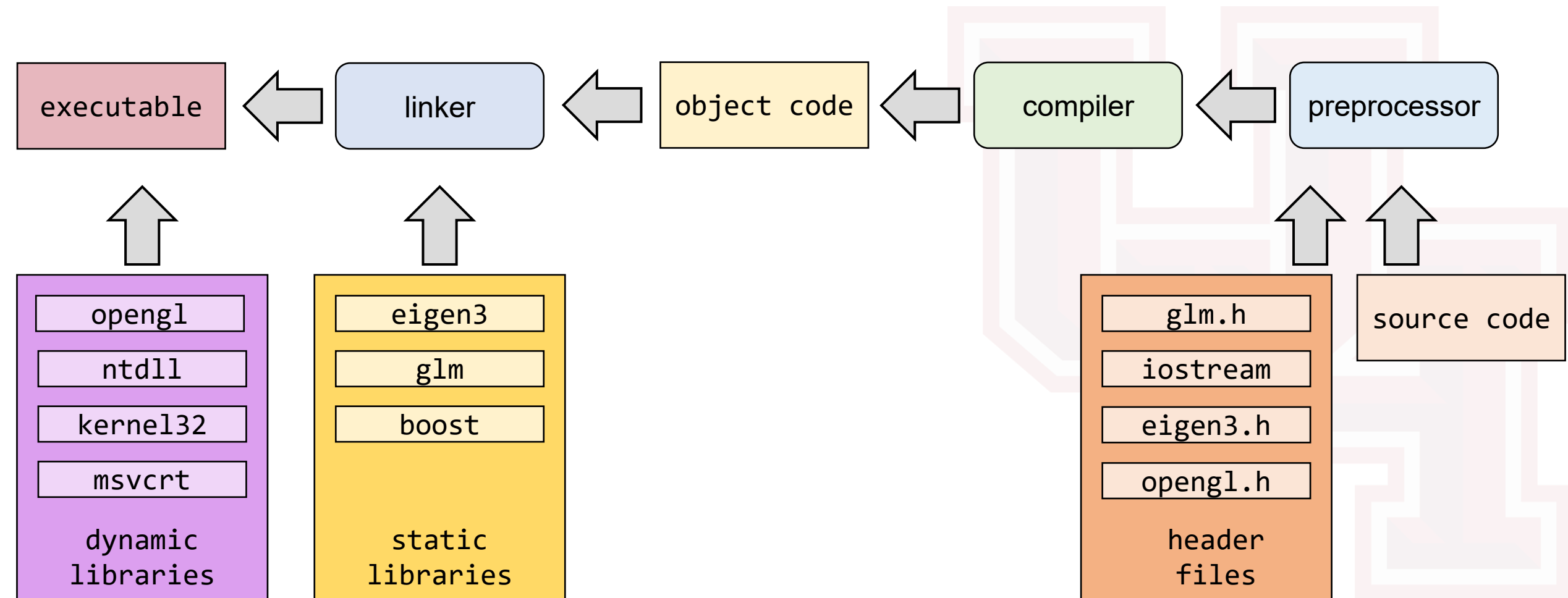
Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory
Department of Electrical and Computer Engineering
Cullen College of Engineering
University of Houston

UNIVERSITY of **HOUSTON** | ECE

C/C++ Build



Compilers

- GNU Compiler collection (gcc/g++)
 - Clang/LLVM (clang/clang++)
 - Open source compiler suites available on all major operating systems
 - Strong support for recent C/C++ language standards
- Microsoft Visual C++ (msvc/cl)
 - Proprietary compiler only available for Windows
 - Poor support for C++ 26, decent support for C++ 23
 - Integrated UI/debugging tools
- Intel oneAPI C++ Compiler (icpx/icx)
 - Proprietary compiler available for all major operating systems
 - Poor support for C++ 26, good support for C++ 23
 - Excellent benchmark/optimization performance, especially with their math libraries



Operating Systems - Recommendations

Windows

Visual C++, GNU, Clang, oneAPI

Visual Studio:

- Easy to install
- Provides good debugging and profiling tools
- Very hard to start a project from scratch

GNU C/C++:

- More difficult to install (use MinGW)
- Need an IDE (CLion, Dev-C++)

Consider using VS with the Intel's oneAPI compiler. Install VS (be sure to click C/C++), then oneAPI.

macOS

GNU, Xcode/Clang, oneAPI

GNU C/C++:

- Install using homebrew
- Very easy to start/use
- Select your IDE: CLion, VS Code, Code::Blocks, Eclipse

Xcode/Clang:

- Default compiler, uses Clang
- Can't use CUDA (uses Metal)

Recommend installing GNU because `std::thread` support is a little spotty. Can't use CUDA on your system, so you'll have to use a server with GNU anyway.

Linux

GNU, Clang, oneAPI

GNU C/C++

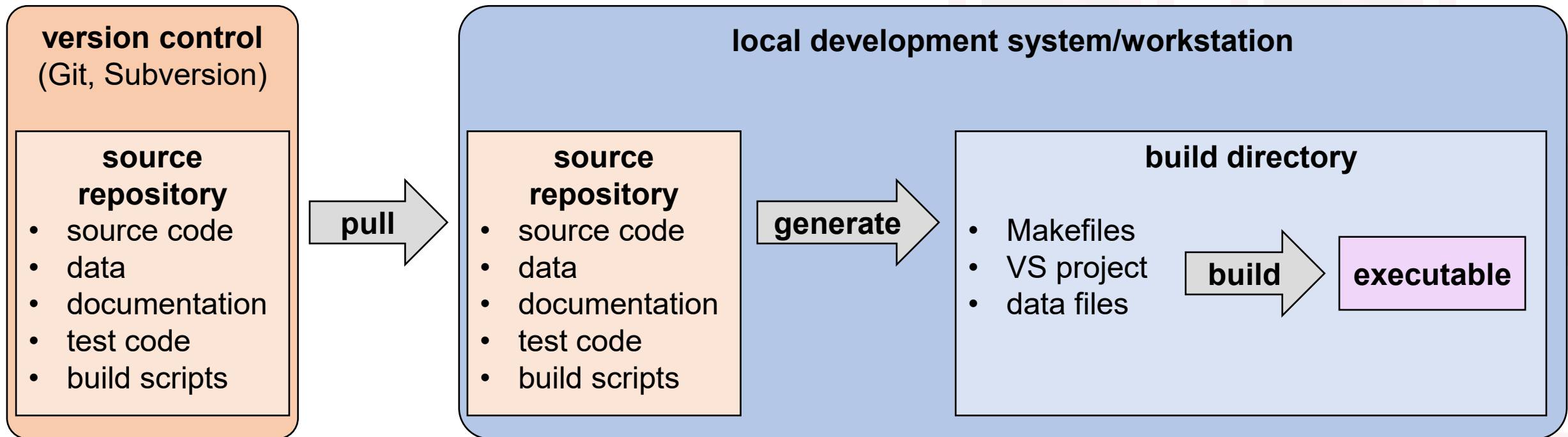
- Install via package manager
- Very easy to start/use
- Select your IDE: CLion, VS Code, Code::Blocks, Eclipse

Intel oneAPI:

- Installation will link GNU
- Generally better benchmarking performance

I prefer CLion with either GNU or oneAPI – usually GNU on workstations and oneAPI on servers.

Working with Good Source Code



Source Code Repository

- myproject
 - CMakeLists.txt
 - src/
 - main_func.cpp
 - other_funcs.cpp
 - myproject.h
 - data/
 - image.jpg
 - spreadsheet.csv
 - docs/
 - Doxyfile
- Put everything needed to build your project from scratch here
- Keep this directory structured
- Make your project easy to build
 - CMakeLists.txt – CMake script describes how to build the project
 - Doxyfile – Doxygen script generates documentation
- Create a separate “build” directory that holds binary files
 - **You should be able to work from different computers by sharing ONLY the source repository**

Build Systems

- Compiler/Linker require:
 - source files to be compiled
 - header files for any external libraries
 - static library locations for linking
- When multiple compilers are used (ex. C + CUDA)
 - source files are passed to a CUDA compiler nvcc
 - CUDA-related source is stripped and compiled into object code
 - remaining source is passed to the C/C++ compiler
 - all object code files are linked at the end
- Anyway, this all gets very complicated and you shouldn't do it manually

Build Automation

- Automation tools create a build system for your desired compiler and IDE
- Find external libraries and header files
- Pass the appropriate parameters to the compiler/linker
- May also create an IDE
- Multiple options
- CMake has flaws but is the closest to an industry standard

Current Standards:

CMake – most popular, bad scripting
cmake.org



Premake – better scripting (Lua)
premake.github.io



Meson – fast for large projects
mesonbuild.com



CMake Scripts (CMakeLists.txt)

```
# specify the CMake version required
cmake_minimum_required(VERSION 3.0)

# create a name for this project
project(myproject)

# “glob” is a term for combining files
file(GLOB_RECURSE SOURCE src/*.cpp src/*.h)
file(GLOB_RECURSE DATA data/*)

# copy data files to the build directory
file(COPY ${DATA} DESTINATION .)

# create an executable from the source code
add_executable(myexe ${SOURCE})
```

- The CMakeLists.txt file is a script describing:
 - what is required to build the project
 - how to build the project
 - what will be created as a result
- The project will be created in the build directory
- If anything goes wrong, you can always delete everything from the build directory and re-run CMake

Using CMake

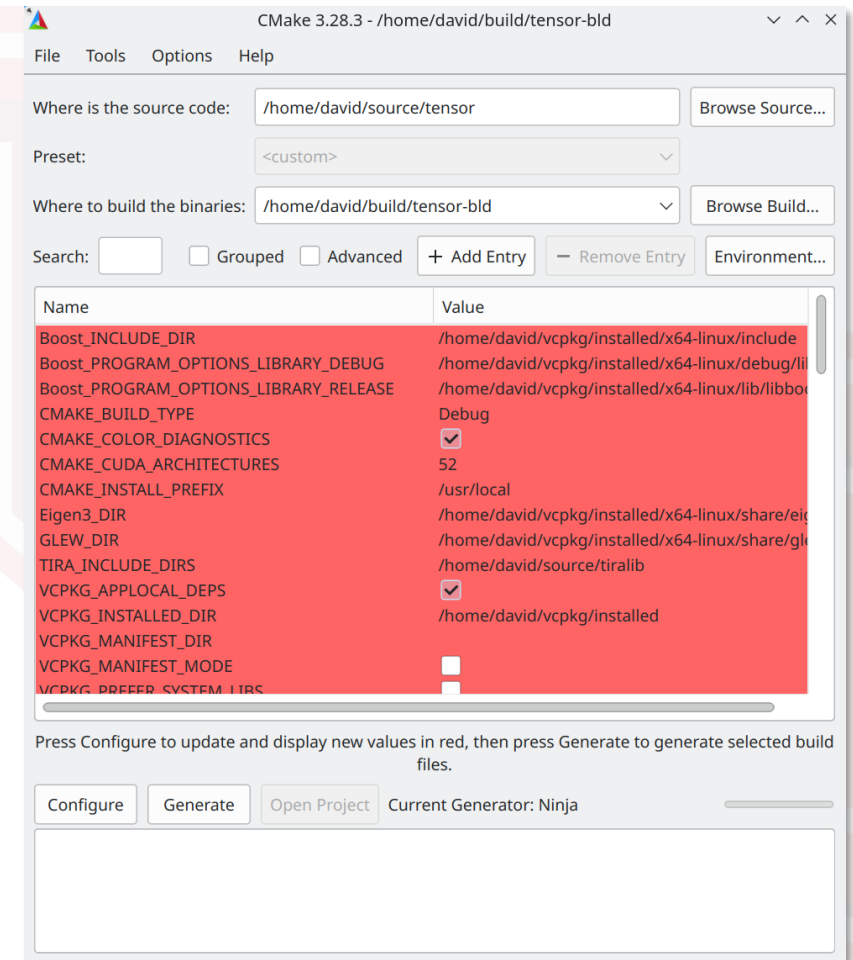
- Pull the repository:

```
git clone https://github.com/name/repo.git
```
- Create your build environment
 - From your source directory:

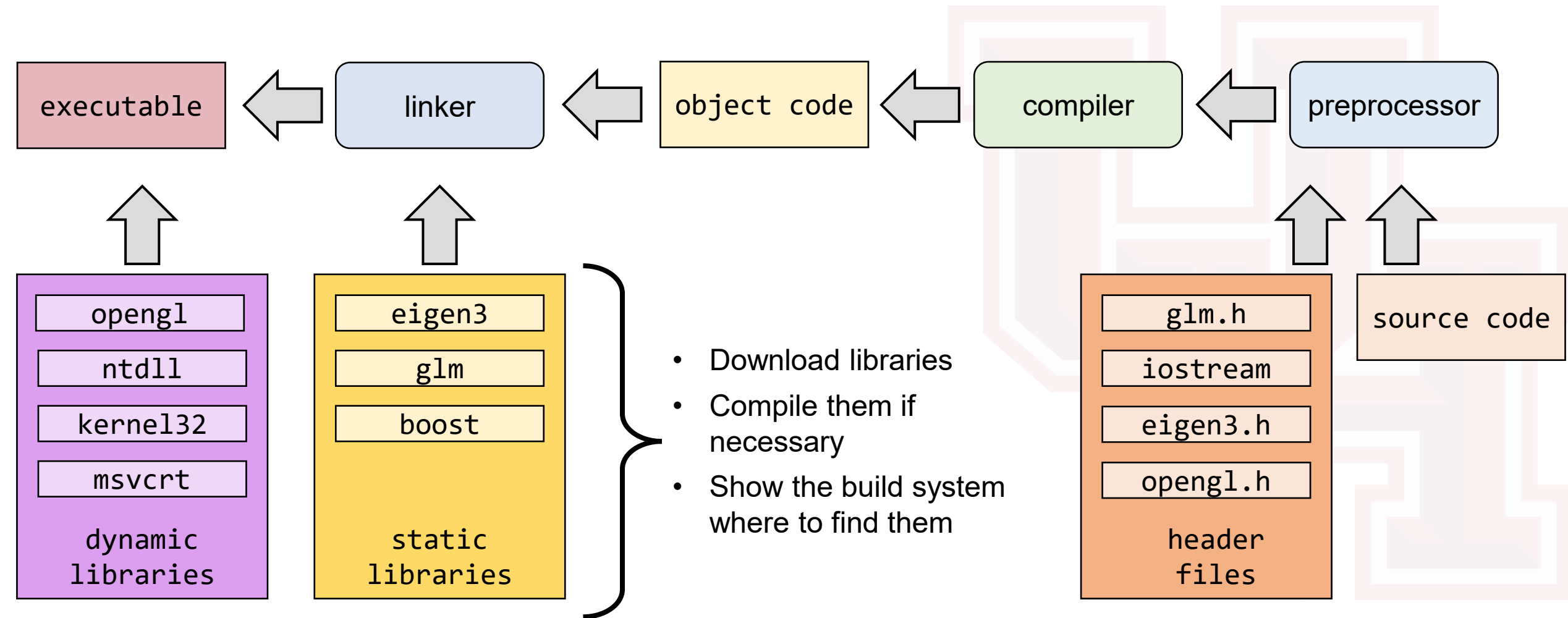
```
cmake -B /path/to/build
```
 - From anywhere:

```
cmake -B /path/to/build -S path/to/source
```
- Compile your project (depends on IDE/compiler)
 - Visual Studio: click “build”
 - GNU:

```
cd /path/to/build  
make
```



Using External Libraries



Package Managers

- Automate installing, updating, and removing software
- This can include software like:
 - static and dynamic libraries
 - executable programs
 - source code and even documentation
- Linux package managers vary with distribution
 - aptitude (apt), Red Hat (rpm), snapcraft
- macOS: homebrew
- Windows doesn't have one BUT
- VCPKG is cross-platform
 - <https://vcpkg.io>
 - Downloads and builds libraries locally
 - Integrates with CMake

vcpkg

- Clone VCPKG as a Git repository

```
git clone https://github.com/microsoft/vcpkg.git
```
- Run the build script (generates a vcpkg executable for your system)

```
cd vcpkg
```

`./bootstrap-vcpkg.bat` (Windows)

or

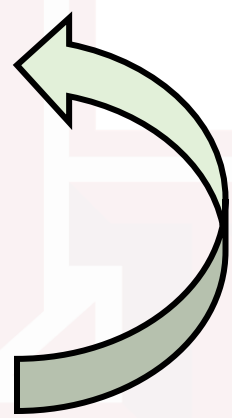
`./bootstrap-vcpkg.sh` (Linux, macOS)
- Install libraries

```
./vcpkg install glfw3
```

```
./vcpkg install glm
```

...

Summary

- Create a source repository
 - Organize your code using directories
 - Use a version tool (probably `git`)
 - Organize revisions, undo mistakes
 - Work on multiple updates
 - Make your repository redundant
 - Dropbox, GitHub, etc.
 - Easy to transfer code between computers
 - Create a `CMakeLists.txt` script
 - Generate the project
 - Edit source code
 - Compile/Run
 - Add libraries as needed
- 

Homework 1

- I created a repository that should be a good starting point for programming projects related to this class:
<https://github.com/STIM-Lab/helloworld>
- This also provides a good opportunity to make sure you have the tools available for the first few programming assignments
- Get it to compile and run using the platform of your choice
- I've heavily commented the code to explain what each step does (including the `CMakeLists.txt` script)
- Upload a screenshot to Canvas when you get it working